GITHUB LINK :- https://github.com/dwivedid544-source/Listing-of-products-Project

Project Deployment Link :- https://listing-of-products-project.vercel.app/

Product Management REST API Using Node.js and Express

Abstract

This paper presents the design and implementation of a Product Management REST API developed using Node.js and Express.js. The system enables efficient management of product data through RESTful endpoints, supporting operations such as data retrieval, filtering, sorting, and insertion. The application uses an in-memory data structure to simulate database functionality. This project demonstrates fundamental backend development concepts and provides a scalable foundation for future enhancements such as database integration and authentication mechanisms.

Keywords

Node.js, Express.js, REST API, Backend Development, Product Management, Web Services

1. Introduction

With the increasing demand for web-based applications, backend systems play a crucial role in managing and processing data efficiently. RESTful APIs have become a standard approach for communication between client and server systems.

This project focuses on developing a lightweight and efficient Product Management API using Node.js and Express.js. The system allows users to perform operations such as retrieving products, filtering based on criteria, sorting results, and adding new products.

---

2. Objective

The primary objectives of this project are:

- To design and implement a RESTful API
- To understand backend development using Node.js
- To implement filtering and sorting mechanisms
- To simulate a product database using in-memory storage

---

3. System Architecture

The system follows a simple client-server architecture:

- **Client Layer:** Sends HTTP requests (GET, POST)
- **Server Layer:** Processes requests using Express.js

- **Data Layer:** In-memory product array

The server acts as an intermediary between the client and the data.

---

4. Technologies Used

Technology	Description
Node.js	JavaScript runtime for server-side execution
Express.js	Web framework for building APIs
CORS	Middleware for cross-origin resource sharing
Path Module	Handles file system paths

---

5. Implementation Details

5.1 Server Setup

The server is built using Express.js and configured to run on port 3000 or a user-defined environment port. Middleware such as CORS and JSON parsing is enabled for smooth request handling.

5.2 Data Model

The application uses an in-memory array to store product data. Each product contains the following attributes:

- ID
- Name
- Price
- Category
- Image URL

5.3 API Endpoints

5.3.1 GET /api/products

Retrieves all products with optional filtering and sorting.

Parameters:

- Category
- Minimum Price
- Maximum Price
- Sorting Order (Ascending/Descending)

5.3.2 POST /api/products Adds a new

product to the system.

Validation:

- All fields must be provided
- Price must be numeric

---

6. Features

- Dynamic product retrieval
- Category-based filtering
- Price range filtering

-
- Sorting by price
- JSON-based API responses

---

7. Results and Output

The API successfully handles requests and returns structured JSON responses. Users can:

- Fetch filtered product lists
- Sort products by price
- Add new products dynamically

The system performs efficiently for small-scale applications.

---

8. Limitations

- No persistent database (data resets on restart)
- Limited to basic CRUD operations (Create and Read only) ● No authentication or security features

---

9. Future Scope

- Integration with databases such as MongoDB
- Implementation of authentication (JWT)
- Addition of Update and Delete operations
- Deployment on cloud platforms
  Implementation of pagination and caching

•

---

10. Conclusion

The Product Management REST API demonstrates the core concepts of backend development using Node.js and Express.js. It provides a simple yet effective implementation of REST principles, including routing, middleware usage, and data handling. This project serves as a strong foundation for developing scalable and production-ready backend systems.

References

[1] Node.js Documentation.

[2] Express.js Documentation. [3] REST API Design Principles.